

Avance 2: PINNs para resolver ecuaciones diferenciales

Jorge Ruiz de Viñaspre

5 de junio de 2026

Resumen

Este informe detalla el desarrollo y la implementación de Redes Neuronales Informadas por la Física (PINNs) para la resolución de un sistema oscilatorio de 4 grados de libertad. Se documenta la transición desde arquitecturas tradicionales hacia redes SIREN con optimizador SOAP, mitigando patologías numéricas como el sesgo espectral y los conflictos de gradientes, logrando una alta fidelidad en la interpolación paramétrica de la respuesta dinámica.

1. Marco Teórico

1.1. PINNs para Resolver Ecuaciones Diferenciales:

Históricamente, la modelación de sistemas físicos ha estado dominada por dos paradigmas disjuntos. Por un lado, los métodos numéricos clásicos (e.g., Elementos Finitos, Diferencias Finitas), los cuales exigen un conocimiento exhaustivo y determinista de las leyes físicas, condiciones de borde y parámetros del sistema, requiriendo además mallas de discretización costosas. Por otro lado, el Aprendizaje Automático Tradicional (*Data-Driven Machine Learning*), que opera como aproximadores universales de caja negra, capaces de inferir patrones desde grandes volúmenes de datos empíricos, pero ignorando por completo la cinemática o dinámica subyacente, lo que los vuelve inútiles para extrapolaciones físicas.

Las Redes Neuronales Informadas por la Física (PINNs), introducidas formalmente por Raissi, Perdikaris y Karniadakis (2019), consolidan el campo del *Scientific Machine Learning* (SciML). Las PINNs conservan la flexibilidad de las redes neuronales para operar con datos dispersos y ruidosos, pero restringen su espacio de soluciones admisibles obligándolas a respetar el operador diferencial (EDO o EDP) que gobierna la fenomenología del problema.

El éxito de las PINNs recae casi exclusivamente en la Diferenciación Automática (*Automatic Differentiation* o AD), el algoritmo subyacente que permite entrenar redes neuronales mediante retropropagación (*Backpropagation*).

A diferencia de la diferenciación numérica empleada en los métodos clásicos (que sufre de errores de truncamiento y exige mallas espaciotemporales) y de la diferenciación simbólica (que sufre de explosión de términos), la AD aplica la regla de la cadena de cálculo diferencial paso a paso sobre el grafo computacional de la red neuronal.

Si una red neuronal $\mathcal{N}$ toma el tiempo t como entrada y predice el desplazamiento u , la velocidad y la aceleración se obtienen analíticamente a través de la red como:

$$\dot{u}(t) = \frac{\partial \mathcal{N}(t; \theta)}{\partial t}, \quad \ddot{u}(t) = \frac{\partial^2 \mathcal{N}(t; \theta)}{\partial t^2} \quad (1)$$

Esto implica que las PINNs son métodos fundamentalmente libres de malla (*mesh-free*). Las derivadas temporales o espaciales son exactas hasta la precisión de la máquina de punto flotante en cualquier coordenada continua del dominio, permitiendo resolver dinámicas complejas sin las inestabilidades de integración numérica como las observadas en esquemas explícitos.

1.2. Aplicación a sistemas oscilatorios

El caso de estudio analizado en este informe radica en un sistema oscilatorio, por lo que se hace hincapié en la fenomenología de las PINNs ante sistemas de esta índole. A la hora de simular modelos oscilatorios, se encuentran carencias por parte de las funciones de activación clásicas de las PINNs (como $\text{ReLU}()$ o $\text{Tanh}()$), favoreciendo soluciones de baja frecuencia. Este fenómeno de sesgo frecuencial se denomina *Spectral Bias*.

El análisis riguroso de este sesgo se estudia mediante el *Neural Tangent Kernel* (NTK). En el límite de ancho infinito, el entrenamiento de una PINN por descenso de gradiente es equivalente a la evolución de un sistema kernel lineal continuo. La teoría del NTK demuestra matemáticamente que la tasa de convergencia espacial o temporal de la red a lo largo de las distintas frecuencias de la función objetivo $\hat{u}(t)$ es proporcional a los valores propios (*eigenvalues*) de la matriz del NTK.

Para activaciones tradicionales, los valores propios decaen exponencialmente frente a las altas frecuencias espaciales o temporales. Esto implica que la red converge más rápido a los componentes constantes o de baja frecuencia, mientras es intrínsecamente “ciega” a las oscilaciones de alta frecuencia.

Las metodologías exploradas en este informe para mitigar esta problemática son la arquitectura **SIREN** y el **Fourier Encoding**. En las SIREN la preactivación lineal es modulada por una función seno escalada por un factor de frecuencia natural ω_0 :

$$y_l = \sin(\omega_0 \cdot \mathbf{W}_l \mathbf{x} + \mathbf{b}_l) \quad (2)$$

Matemáticamente, la composición de funciones seno a través de múltiples capas ocultas genera un ensanchamiento explosivo del ancho de banda del NTK. La red neuronal deja de ser un aproximador local pasivo y se convierte en un operador que contiene en sus gradientes una expansión armónica generalizada (análoga a una serie de Fourier anidada y no lineal), permitiéndole capturar las frecuencias naturales del sistema dinámico.

Una alternativa a las SIRENs para mitigar la incapacidad de las redes neuronales estándar de capturar dinámicas de alta frecuencia, es el Mapeo de Características de Fourier (*Fourier Feature Encoding*). Esta técnica preserva la arquitectura clásica de la red, utilizando activaciones tradicionales como $\text{Tanh}()$, pero transforma las coordenadas de entrada de baja dimensión (por ejemplo, el tiempo t o las coordenadas espaciales $\mathbf{x}$) proyectándolas sobre una base hiperespacial de funciones periódicas antes de que ingresen a la primera capa oculta. Matemáticamente, este mapeo se define como:

$$\gamma(\mathbf{v}) = [\cos(2\pi \mathbf{B}\mathbf{v}), \sin(2\pi \mathbf{B}\mathbf{v})]^T \quad (3)$$

Donde $\mathbf{v}$ es el vector de entrada original y $\mathbf{B}$ es una matriz de proyección cuyos elementos se muestrean aleatoriamente desde una distribución gaussiana isotrópica $\mathcal{N}(0, \sigma^2)$. Desde la perspectiva del NTK, la introducción de esta capa de codificación traslada artificialmente el espectro de los valores propios del kernel computacional.

1.3. PINNs Paramétricas

Las PINNs Paramétricas representan una evolución de la formulación original donde el modelo neuronal recibe como entradas no solo las coordenadas espacio-temporales (x, t) , sino también los parámetros físicos u operacionales que definen el sistema (tales como coeficientes de amortiguamiento o la frecuencia de una carga externa ω). En lugar de resolver una EDO aislada para un escenario fijo, el optimizador aprende toda la familia de soluciones posibles del operador diferencial, convirtiendo a la red en un meta-modelo sustituto (*surrogate solver*).

1.4. *Time-Marching PINNs*

Por su parte, las PINNs *Time-Marching* (o de aproximación marchante) constituyen una estrategia de descomposición de dominios diseñada para evitar el colapso de causalidad que sufren las redes estándar al simular eventos en horizontes temporales extensos. En lugar de optimizar la solución global simultáneamente, la simulación se particiona en sub-ventanas temporales (por ejemplo, $\Delta t = 0,02$ s), inyectando los estados finales de manera autorregresiva como nuevas condiciones iniciales.

1.5. Inestabilidad de Pérdida en entrenamientos extensos

Durante las etapas tardías del entrenamiento de una red neuronal, es común observar un régimen de oscilación estocástica en la función de pérdida. Esta patología ocurre porque la tasa de aprendizaje (*learning rate*) se mantiene fija. A medida que los pesos se aproximan a las mínimas locales en la superficie de pérdida, el paso dictado por el optimizador se vuelve demasiado grande para la sutil curvatura local. La implementación de un programador de la tasa de aprendizaje (*Learning Rate Scheduler*), tal como `ReduceLROnPlateau`, mitiga críticamente este comportamiento al “enfriar” de forma progresiva el proceso de optimización.

2. Objetivos Fase 2

En la primera fase del proyecto se obtuvo un modelo de PINN simple la cual presentaba carencias en extrapolación temporal y utilidad, debido a que ésta sólo servía con un conjunto de parámetros específicos de la EDO a resolver, aparte de representar un contexto operacional no interesante. Todo esto llevaba a un resultado inferior a *solvers* tradicionales en todos los ejes de análisis. Debido a estas carencias se plantearon los siguientes objetivos para la fase 2:

- Aplicación sobre un contexto operacional de mayor complejidad y utilidad, perteneciente al área ingenieril o física.
- Resolver el problema de extrapolación respecto al dominio de entrenamiento.

- Implementar un paradigma en el cual la PINN presente (en un principio de manera teórica) una mejora por sobre *solvers* tradicionales.

3. Desarrollo Investigativo

El primer paso de desarrollo fue cambiar el contexto operacional sobre el cual se está aplicando el concepto de las PINNs, buscando un balance entre utilidad o relevancia de la implementación y su dificultad. Durante la primera fase de desarrollo se encontró la publicación de Wenhao Zhang, et. al. [7]. En este se plantea la aplicación de PINNs para tres contextos de análisis estructural: un sistema de 2 grados de libertad (**DOF**), un modelo de marco de edificio de 4 pisos, con 4 DoF, y una viga colgante con 6 DoF. Se decidió implementar el modelo de marco de edificio de 4 pisos, siendo considerado como algo logvable dadas las condiciones de trabajo.

3.1. Contexto operacional

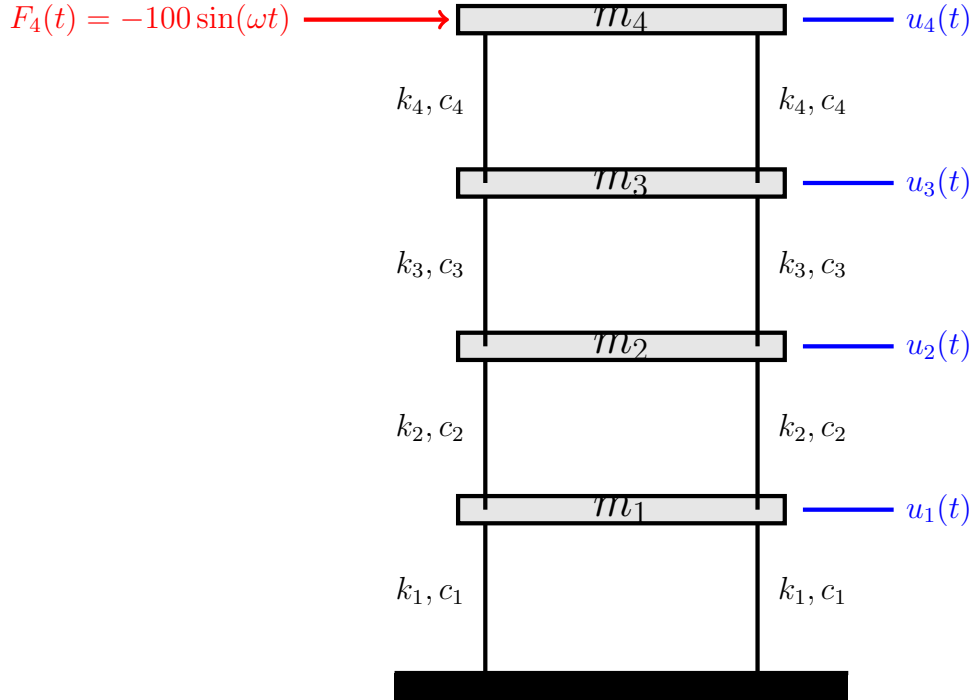


Figura 1: Modelo esquemático del edificio de corte de 4 grados de libertad ($N_{dof} = 4$). u_i representa la traslación desde el equilibrio del i -ésimo piso

El sistema consiste en un modelamiento simplificado utilizado en ingeniería civil para simular la respuesta de estructuras ante forzamientos externos. Este se modela como un sistema de osciladores amortiguados acoplados, el cual se plantea de forma matricial con la siguiente ecuación y componentes:

$$\mathbf{M}\ddot{\mathbf{u}}(t) + \mathbf{C}\dot{\mathbf{u}}(t) + \mathbf{K}\mathbf{u}(t) = \mathbf{F}(t)$$

El Vector de Estado ($\mathbf{u}$) Representa los desplazamientos horizontales de cada losa respecto a su posición de equilibrio en la base. Sus derivadas temporales correspon-

den a las velocidades ($\dot{\mathbf{u}}$) y aceleraciones ($\ddot{\mathbf{u}}$).

$$\mathbf{u}(t) = \begin{bmatrix} u_1(t) \\ u_2(t) \\ u_3(t) \\ u_4(t) \end{bmatrix}$$

La Matriz de Masa ($\mathbf{M}$) En un modelo de edificio de corte idealizado, se asume que toda la masa de la estructura está concentrada (lumped mass) a nivel de las losas, y que las columnas no tienen masa. Como el movimiento de la masa 1 no genera fuerzas de inercia directamente sobre la masa 2, la matriz es estrictamente diagonal. Con $m = 80,0$ toneladas por piso, la matriz queda:

$$\mathbf{M} = \begin{bmatrix} m_1 & 0 & 0 & 0 \\ 0 & m_2 & 0 & 0 \\ 0 & 0 & m_3 & 0 \\ 0 & 0 & 0 & m_4 \end{bmatrix} = \begin{bmatrix} 80 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 \\ 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 80 \end{bmatrix}$$

La Matriz de Rigidez ($\mathbf{K}$) Esta es la matriz más compleja y refleja cómo los pisos interactúan elásticamente entre sí. Se construye analizando las fuerzas de restitución de las columnas (actuando como resortes con constante $k = 400,000$ N/m). Para el piso 1: Está conectado al suelo (desplazamiento 0) y al piso 2. La fuerza elástica neta es $-k_1 u_1 + k_2(u_2 - u_1)$. Agrupando términos respecto a u , obtenemos una rigidez propia de $(k_1 + k_2)$. Para los pisos intermedios (2 y 3): Sienten la resistencia del piso inferior y del piso superior. Para el techo (piso 4): Solo está conectado al piso de abajo, por lo que su rigidez propia es simplemente k_4 . Esto genera una matriz tridiagonal simétrica, característica fundamental de los sistemas acoplados en cadena:

$$\mathbf{K} = \begin{bmatrix} k_1 + k_2 & -k_2 & 0 & 0 \\ -k_2 & k_2 + k_3 & -k_3 & 0 \\ 0 & -k_3 & k_3 + k_4 & -k_4 \\ 0 & 0 & -k_4 & k_4 \end{bmatrix}$$

Sustituyendo el valor constante k :

$$\mathbf{K} = 400,000 \cdot \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix}$$

La Matriz de Amortiguamiento ($\mathbf{C}$) Calcular el amortiguamiento intrínseco de cada elemento estructural es físicamente inviable. En dinámica estructural, se utiliza el modelo de Amortiguamiento Clásico de Rayleigh, el cual asume que la matriz $\mathbf{C}$ es una combinación lineal proporcional a la masa y a la rigidez del edificio. Usando los coeficientes de proporcionalidad ($\alpha = 0,05$ y $\beta = 0,01$), la matriz se define directamente como:

$$\mathbf{C} = \alpha \mathbf{M} + \beta \mathbf{K}$$

El Vector de Fuerza Externa ($\mathbf{F}$) Define las excitaciones dinámicas aplicadas al sistema. Como la carga armónica $-100 \sin(\omega_{fuerza} t)$ está aplicada exclusivamente en

la losa superior, los primeros tres elementos del vector son nulos.

$$\mathbf{F}(t) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -100 \sin(\omega_{fuerza} t) \end{bmatrix}$$

Para llevar a cabo la validación de las PINN's utilizadas, se utiliza la solución obtenida usando *solveivp* de *scipy* con el método *RK45*.

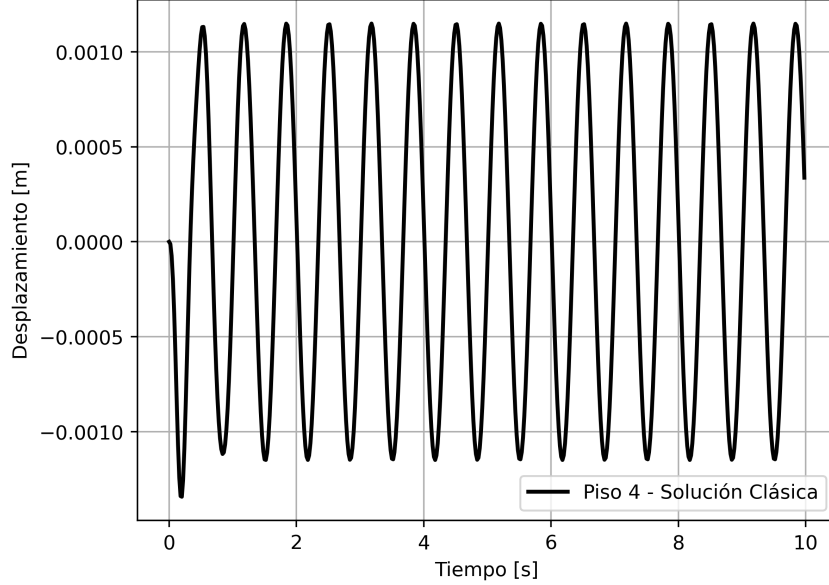


Figura 2: Solución con Scipy para el cuarto piso sobre 10 segundos con $\omega_{fuerza} = 2.75$ Hz

3.2. Modelo Inicial y Extrapolación Temporal

Buscando la aplicabilidad de las PINNs, se integraron los conceptos de PINNs paramétricas, las cuales toman como entrada tanto el vector posición/tiempo como un set de parámetros específicos del problema $\vec{\omega}$. Entrenar la red utilizando varias combinaciones distintas de $\vec{\omega}$ abre la posibilidad de interpolar configuraciones específicas no presentes en el entrenamiento.

Se implementó un modelo *Fully Connected* inspirado en el modelo de 4 DoF. El dominio de entrenamiento consta de una matriz de dimensiones $N_{muestras} \times 11$, estructurada como: $[t_{abs}, \Delta t, u_0, u_1, u_2, u_3, v_0, v_1, v_2, v_3, \omega_{fuerza}]$. Para el cual t_{abs} representa un vector de tiempo "global", añadido para asegurar la continuidad del forzamiento aplicado respecto al tiempo. Δt es el timestep utilizado para el marchado temporal de la Pinn,

La logica de la matriz de entrada para la red es la siguiente: Para algún tiempo arbitrario t_{abs} , utilizando condiciones iniciales de desplazamiento $\vec{u}$ y velocidad $\vec{v}$, la red busca resolver el sistema sometido a una fuerza oscilatoria con frecuencia ω_{fuerza} . Se entregan 5000 combinaciones distintas de estos parámetros, acotando los posibles valores según limitaciones físicas del sistema. Dentro del paradigma de *PINN's Paramétricas*, ω_{fuerza} representa el parámetro operacional "libre". Aprovechando la tendencia periódicamente oscilatoria del sistema, se mapea circularmente t_{abs} , de

manera tal que siempre se encuentre en el rango $[0, \text{Periodo Estimado}]$, permitiendo que la red pueda extrapolar temporalmente de manera indefinida.

Entrenando la red sobre 14,000 épocas se identificaron oscilaciones en la función de pérdida. Con números altos de épocas, la red llega eventualmente a un mínimo local en el que se queda “rebotando”. Este problema se resolvió utilizando un *Scheduler* para el *Learning Rate* (`ReduceLROnPlateau`), cuyo efecto se puede identificar por inspección en la siguiente Figura.

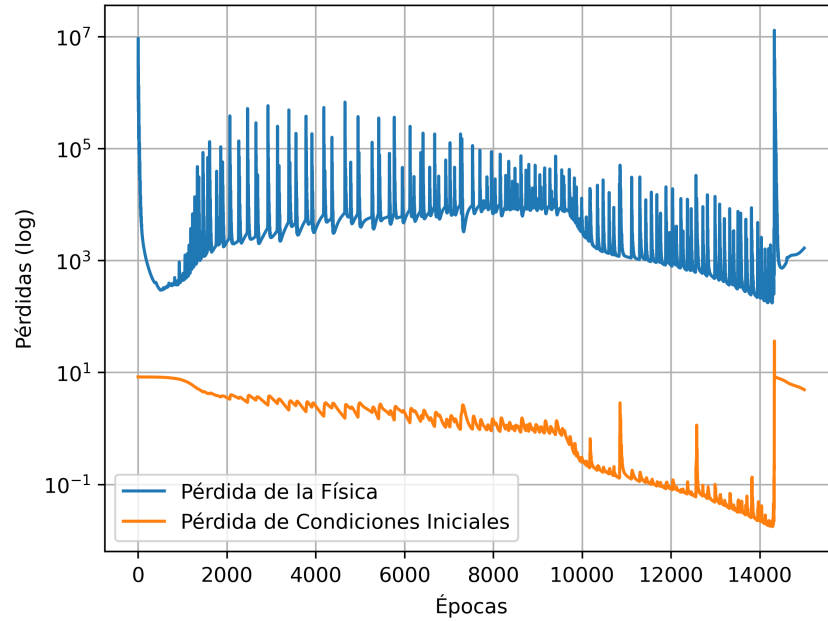


Figura 3: Loss vs Epochs para arquitectura inicial sin Scheduler. Notamos oscilación en la pérdida para números altos de epochs

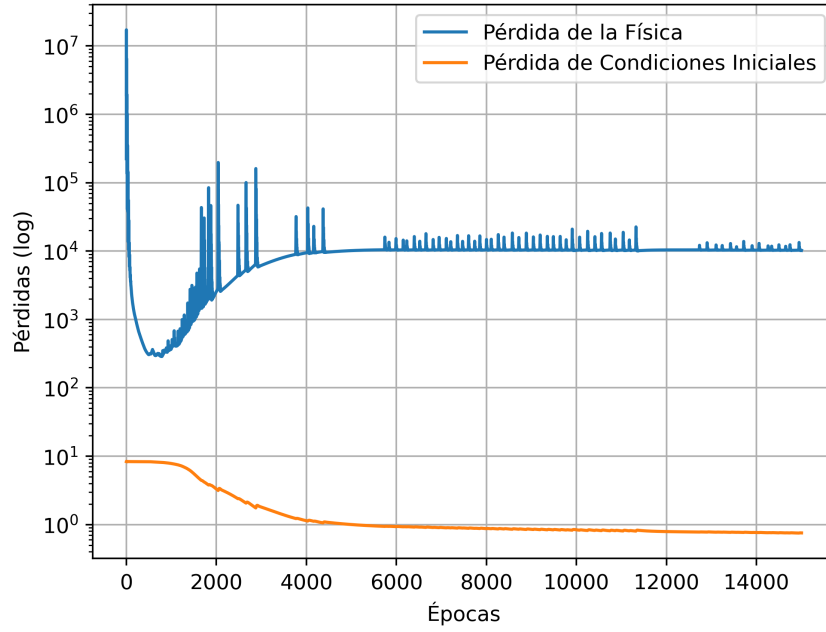


Figura 4: Loss vs Epochs para el mismo modelo, ahora utilizando `ReduceLROnPlateau(factor=0.5, patience=80)`. Notamos como las oscilaciones se ven en gran medida amortiguadas.

El funcionamiento del Scheduler es monitoreado por medio del graficado del Learning rate a lo largo de las epochs:

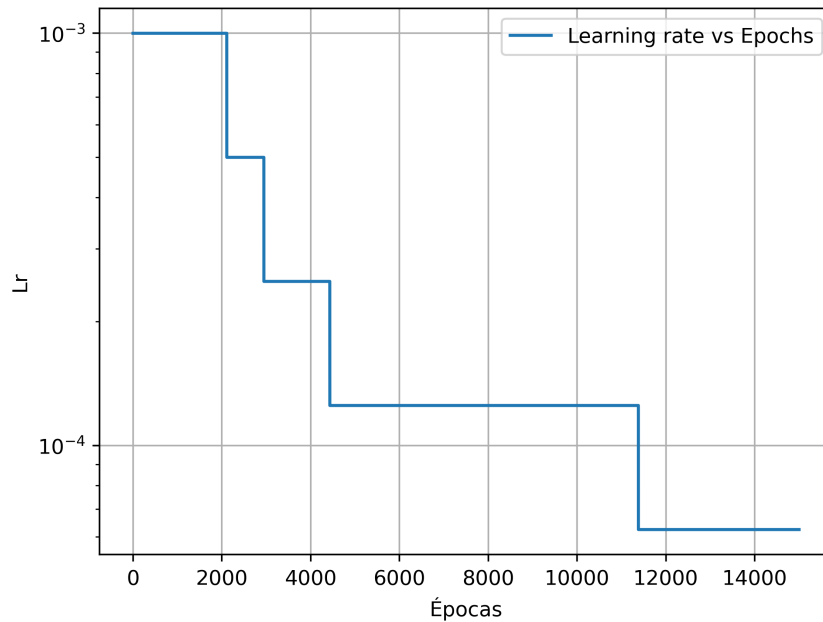


Figura 5: Learning Rate vs Epochs para modelo Tanh, según Scheduler `ReduceLROnPlateau(factor=0.5, patience=80)`.

Se identifica cualitativamente la alta correlación entre la reducción reactiva del Learning Rate con la disminución de la amplitud de los *peaks* de oscilaciones en la pérdida a lo largo de las epochs en el gráfico 4.

Considerando estos resultados, se mantiene el uso del Scheduler a lo largo de las iteraciones del modelo implementadas. Para cuantificar la pérdida dentro del entrenamiento se calcula el residuo de la ecuación diferencial $loss_f$ y el residuo de las condiciones iniciales $loss_{ic}$, planteando una pérdida total por epoch equivalente a:

$$Loss_{total} = loss_f + \lambda_{ic} \cdot loss_{ic}$$

En el cual λ_{ic} es un hiperparámetro que aumenta la relevancia relativa de las condiciones iniciales para la red.

El proceso de desarrollo del modelo se realizó sobre la corrección y mejoramiento iterativo de modelos con distintas arquitecturas:

1. Modelo Tanh
2. Modelo Fourier Encoding
3. Modelo SIREN

El modelo Tanh hace referencia a la arquitectura utilizada durante la fase 1, siendo aplicada al nuevo contexto operacional. Esta red consta de 4 capas de 128 neuronas con funciones de activación $\tanh()$. Tras obtener una pérdida total de $1,79 \cdot 10^4$ en el entrenamiento (véase grafico 4), se evalúa el modelo, el cual se nota incapáz de converger a la solución esperada, obteniendo un error promedio absoluto $MAE = 4,87 \cdot 10^{-2} m$.

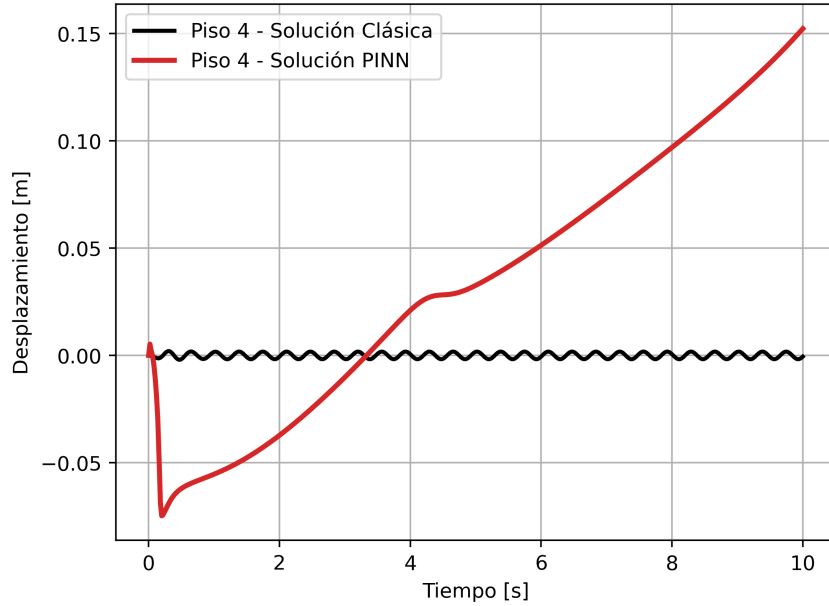


Figura 6: Resultado de modelo Tanh para el 4o piso sobre 10 segundos con $\omega_{fuerza} = 2.75$ Hz. Este gráfico es representativo de la tendencia en el resto de los pisos.

La principal preocupación que genera este resultado es la total ausencia de cualquier tendencia oscilatoria, además de errores enormes en la amplitud del fenómeno. Esto nos lleva a la búsqueda de arquitecturas distintas que capturen la lógica del sistema.

3.3. Modelos Fourier Encoding y SIREN

Buscando mejorar el desempeño del modelo, tras búsqueda respecto a posibles soluciones de PINN's ya utilizadas y estudiadas en la literatura, se encuentran dos posibles arquitecturas candidatas para resolver el problema: Fourier Encoding y SIREN. Se lleva a cabo una implementación preliminar de ambas, eligiendo seguir la línea de desarrollo con mejor desempeño.

El modelo Fourier Encoding consta de una capa inicial, en la cual se lleva a cabo la codificación trigonométrica de los componentes t_{abs} , Δt y ω_{fuerza} de la matriz de datos de entrada. Esta codificación se entrega junto a las condiciones iniciales a la red, constando de 4 capas con activación $\text{Tanh}()$, de 128 neuronas. La distribución de frecuencias para el encoding sigue una distribución gaussiana, introduciendo el hiperparámetro σ . Se lleva a cabo el entrenamiento de la red con los siguientes parámetros relevantes:

Tabla 1: Hiperparámetros para red Fourier Encoding.

Parámetro	Valor
Timestep	0.02
Learning rate Inicial	0.001
Rango ω	$[3\pi, 7\pi]$
Epochs	15000
λ_{ic}	10000
Scheduler	ReduceLROnPlateau(factor=0.5, patience=80)
σ	15

Se obtiene una pérdida final de $\approx 2,11 \cdot 10^3$, representado en el siguiente gráfico:

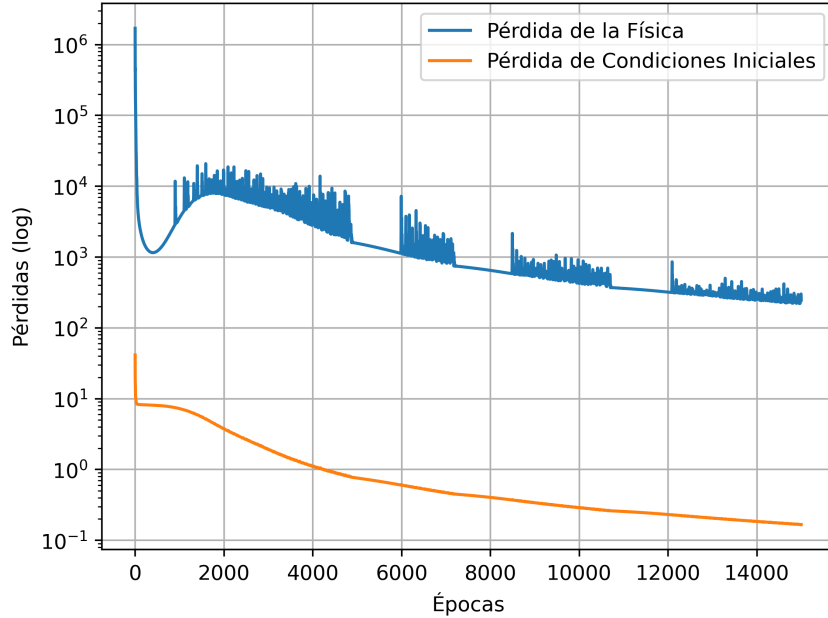


Figura 7: Loss vs Epochs para modelo Fourier Encoding

Tras el entrenamiento, obtenemos un $MAE = 3,23 \cdot 10^{-2}$, presentando un comportamiento visible en la siguiente figura.

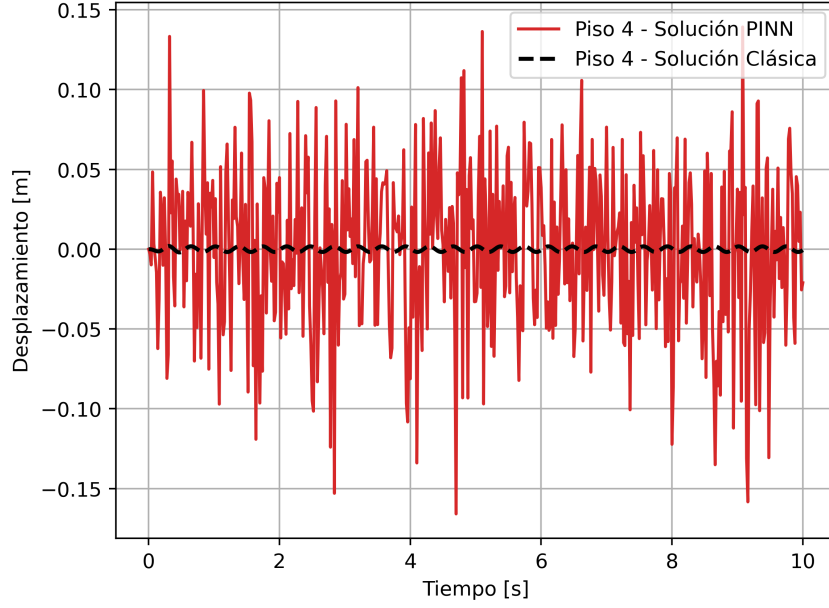


Figura 8: Modelado de oscilación para piso 4 con modelo Fourier Encoding sobre 10 segundos con $\omega_{fuerza} = 2.75$ Hz. El comportamiento de los otros pisos es similar.

Se nota que, a diferencia del modelo Tanh, la red logra plantear un comportamiento que por lo menos logra ser oscilatorio, pero manteniendo errores en la amplitud y frecuencia que sobrepasan el orden de magnitud.

La red SIREN consta de funciones de activación sinusoidales, presentando un hiperparámetro ω_0 a optimizar, el cual dicta la tendencia de las frecuencias de las funciones de activación en la red. Se lleva a cabo el entrenamiento de la red SIREN utilizando los siguientes hiperparámetros:

Tabla 2: Hiperparámetros de red SIREN.

Parámetro	Valor
Timestep	0.02
Learning rate Inicial	0.001
Rango ω	$[3\pi, 7\pi]$
Epochs	15000
λ_{ic}	10000
Scheduler	ReduceLROnPlateau(factor=0.5, patience=80)
ω_0	5

Se obtienen las siguientes curvas de pérdida y de oscilación, con un $MAE_{Global} = 1,067 \cdot 10^{-3}$:

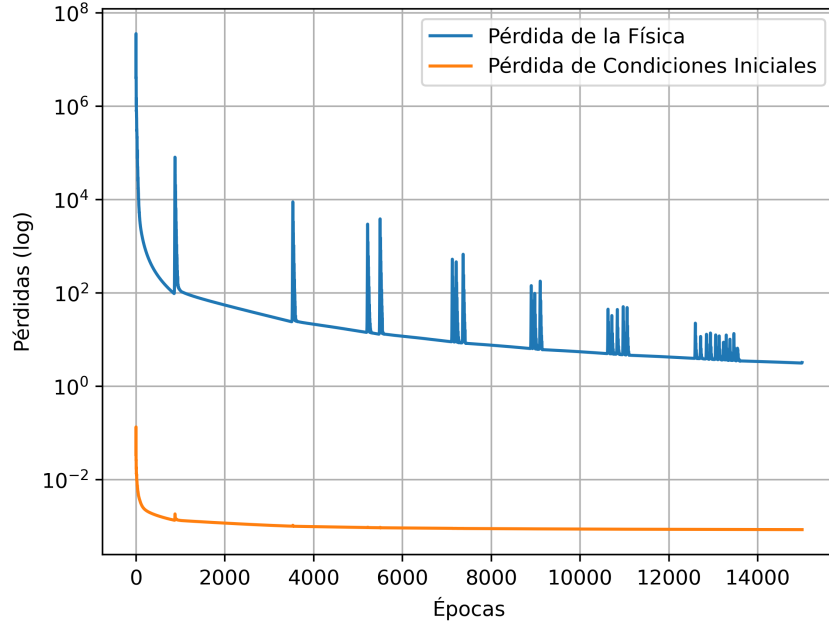


Figura 9: Loss vs Epochs de arquitectura SIREN

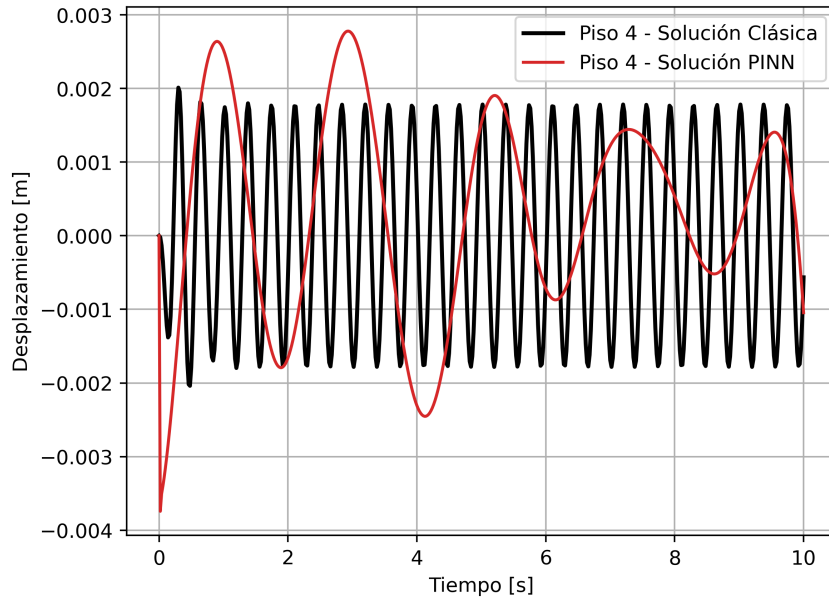


Figura 10: Resultado de SIREN preliminar para piso 4 sobre 10 segundos con $\omega_{fuerza} = 2.75$ Hz.

De inmediato se identifica un desempeño mejor que en el modelo Fourier Encoding, identificando fallos menores en la amplitud y una salida de red de frecuencia muy baja, considerando que esta es modulada por el hiperparámetro ω_0 , se procede a utilizar *Optuna* para la optimización de hiperparámetros inicial. En esta se dejan a optimizar los siguientes hiperparámetros y reportamos sus respectivos valores óptimos tras 29 iteraciones.

1. Learning rate $\approx 3,1928 \cdot 10^{-3}$
2. $\omega_0 \approx 16,0362$
3. $\lambda_{ic} = 1000$
4. Hidden Features = 128

Se actualizan los hiperparámetros obtenidos previamente con los resultados de optuna, también se aumenta las Epochs a 50000 y $N_{muestras}$ de 5000 a 10000, obteniendo el siguiente resultado visual, con un $MAE_{Global} = 7,94 \cdot 10^{-4}$.

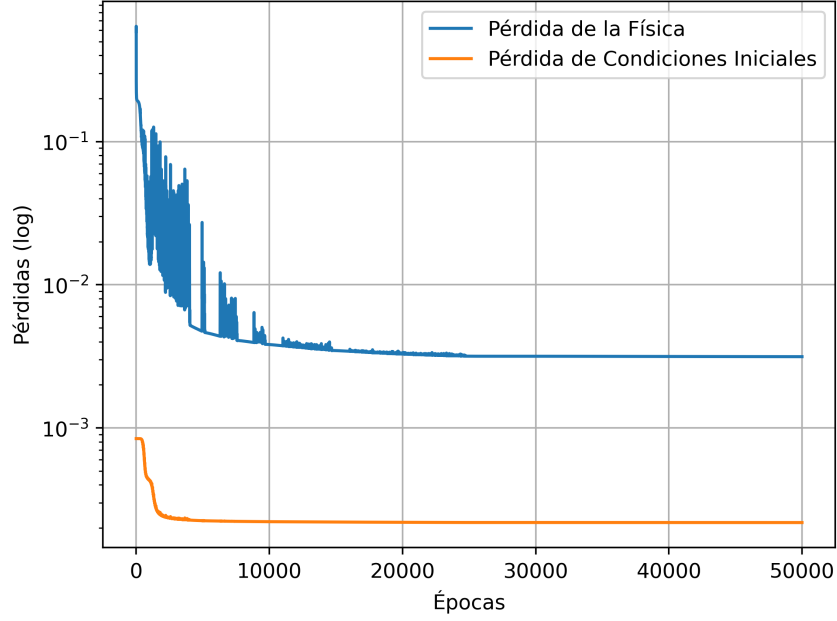


Figura 11: Loss vs Epochs para SIREN post optimización con OPTUNA

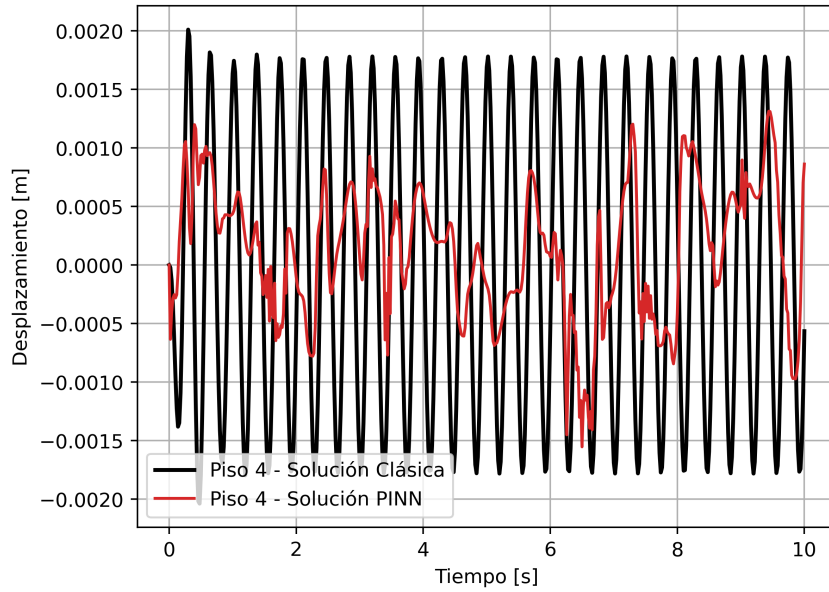


Figura 12: Solución de red SIREN con hiperparámetros optimizados con OPTUNA sobre 10 segundos con $\omega_{fuerza} = 2.75$ Hz.

Se identifica una mayor consistencia en términos de la frecuencia de la salida de la red, en comparación a la solución numérica clásica, indicando un éxito parcial en la optimización de ω_0 . De todas maneras, se identifican claras carencias en la amplitud del resultado. Debido se lleva a cabo una búsqueda sobre el estado del arte de las PINN's. En este proceso se encuentra la publicación por Siavash Khodakaramia et. al. [2], en la cual se presenta la siguiente figura comparando el desempeño de distintos optimizadores para PINN's en sistemas oscilatorios:

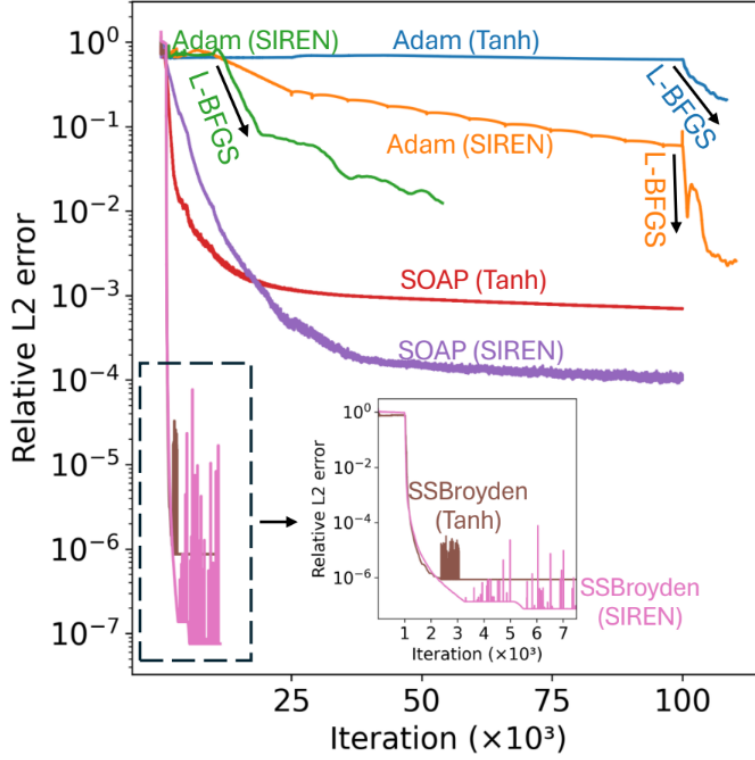


Figura 13: Figura extraida de Spectral bias in physics-informed and operator learning: Analysis and mitigation guidelines, con pie de nota: *Figure 3: KdV equation: PINN training history with different optimizers (Adam, L-BFGS, SOAP, and SS-Broyden) and activation functions (Tanh, and SIREN) for KdV equation. The sub-figure shows the zoomed-in training history for the SS-Broyden optimizer.*

Este resultado inspira la utilización de SS-Broyden para la arquitectura SIREN, la cual no se lleva a cabo, debido la implementación de este optimizador está hecha para jax en vez de pytorch, implicando un costo de tiempo demasiado alto. Se decide utilizar la siguiente opción: SOAP [6], la cual debería presentar mejoras sustanciales en el desempeño del modelo. La principal diferencia en respecto a Adam es que SOAP calcula partes de la matriz Hessiana de la superficie, contabilizando parcialmente su curvatura, mientras que Adam es solamente de primer orden. Esto le permite sortear minimas locales que dejarían a Adam estancado. Junto a la implementación de SOAP, se introdujo un *factor de escala* a la red, este cumple la función de reducir la carga sobre los pesos y gradientes de manejar numeros con órdenes de magnitud muy distintos (como la masa con la amplitud de oscilación). Este factor se aplica al resultado en el `forward()` de la red, multiplicándolo por 10^{-3} , buscando amortiguar la explosión o desvanecimiento de gradientes en las capas finales. La red se vuelve a

someter a un proceso de optimización de hiperparámetros, tras 47 trials se obtiene la siguiente combinación para el entrenamiento final:

Tabla 3: Parámetros y configuración de SIREN SOAP.

Parámetro	Valor
Timestep	0.02
Learning rate Inicial	$2,0970 \cdot 10^{-4}$
Rango ω	$[3\pi, 7\pi]$
Epochs	50000
λ_{ic}	504.5318
Scheduler	ReduceLROnPlateau(factor=0.5, patience=200)
ω_0	18,2613

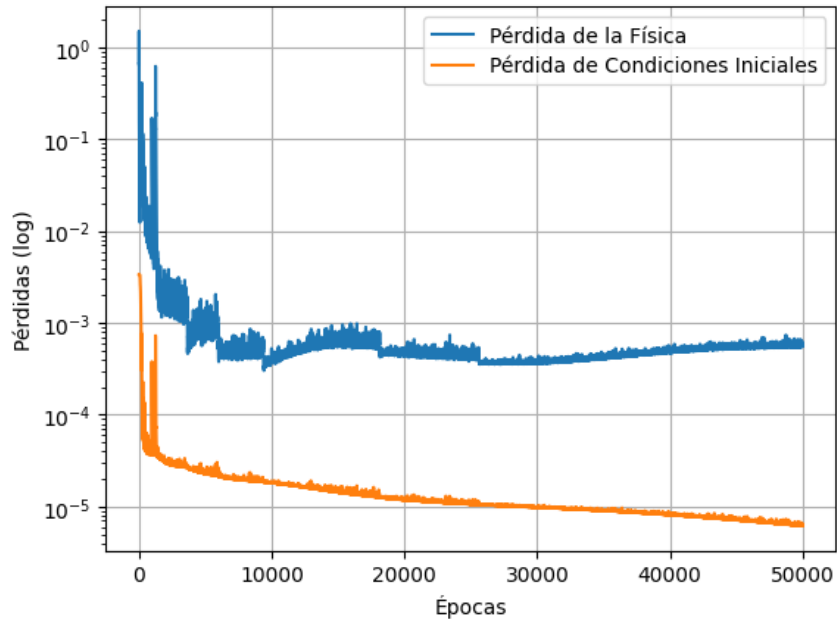


Figura 14: Loss vs Epochs para SIREN con optimizador SOAP

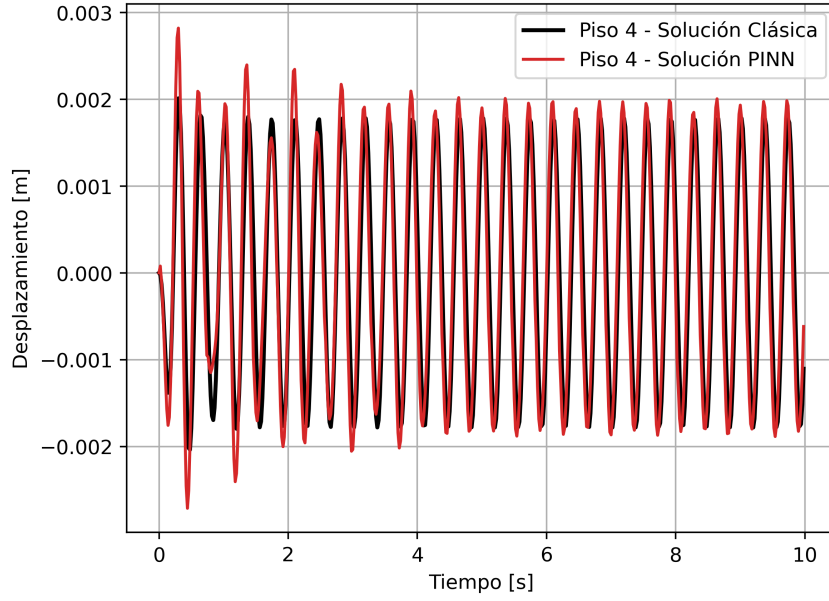


Figura 15: Resultado para 4o piso de red SIREN usando optimizador SOAP sobre 10 segundos con $\omega_{fuerza} = 2.75$ Hz.

Se obtiene un $MAE_{global} = 3,31 \cdot 10^{-4}$. Se perciben todavía errores en términos de la amplitud, pero sustancialmente menores a los vistos previamente. Para evaluar la extrapolación temporal y la interpolación de resultados para valores distintos de ω_{fuerza} , se re-evalúa la red para distintos valores.

Probamos evaluando la red para un tiempo total de 100 segundos, obteniendo el siguiente gráfico:

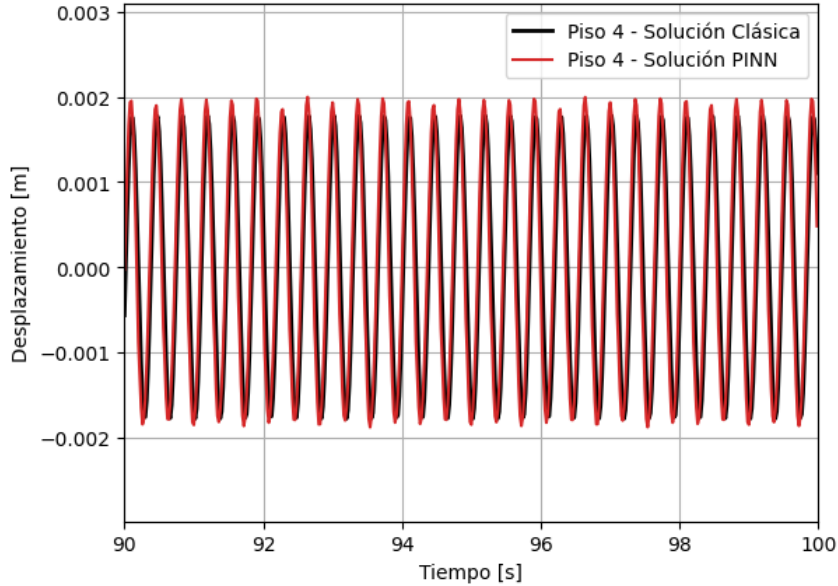


Figura 16: Solución de la red gráfíada para intervalo [90;100] segundos para 4o piso con $\omega_{fuerza} = 2,75$ Hertz.

Se identifica que la red no produce variaciones físicamente incorrectas en la energía, apegándose a la tendencia global de la solución numérica para avances

de tiempo extendidos, comprobando el correcto funcionamiento del *Time Marching* en la PINN.

Para comparar el costo computacional de la evaluación de la red, se miden los tiempos de evaluación de la red vs los tiempos de solución de `solve_ivp` para (1, 10, 20, 100) segundos. Realizando un ajuste lineal obtenemos los siguientes ratios de costo de evaluación:

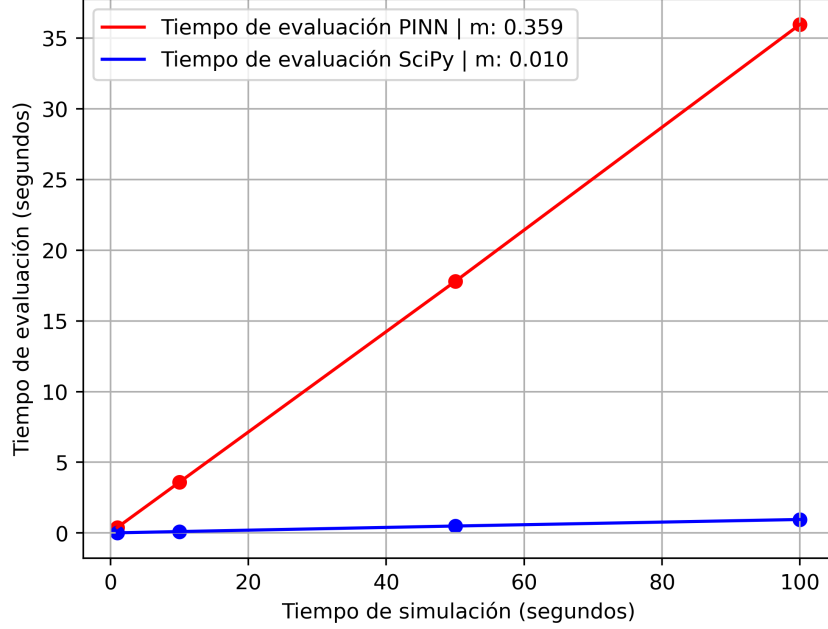


Figura 17: Tiempo de valuación de la red vs tiempo de solución de `solve_ivp`.

Indicando que la red PINN tiene un costo de evaluación de complejidad $O(n)$ ≈ 40 veces mayor que `solve_ivp`, plantando baja aplicabilidad en esta forma.

Evaluando la red en ω_{fuerza} fuera del rango de entrenamiento (1.5 a 3.5 Hz), se identifican fallos en la capacidad de la red de resolver el sistema, mientras que si la frecuencia se mantiene dentro del rango de entrenamiento, la red logra mantener un comportamiento estable.

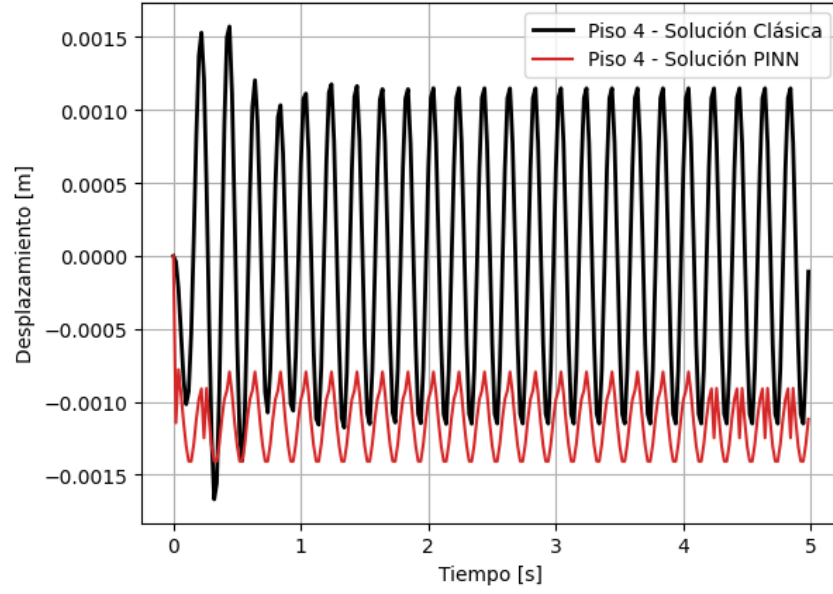


Figura 18: Resultado para la red en 4o piso con $\omega_{fuerza} = 5$ (fuera de rango).
 $MAE_{piso} \approx 7,08 \cdot 10^{-4}$

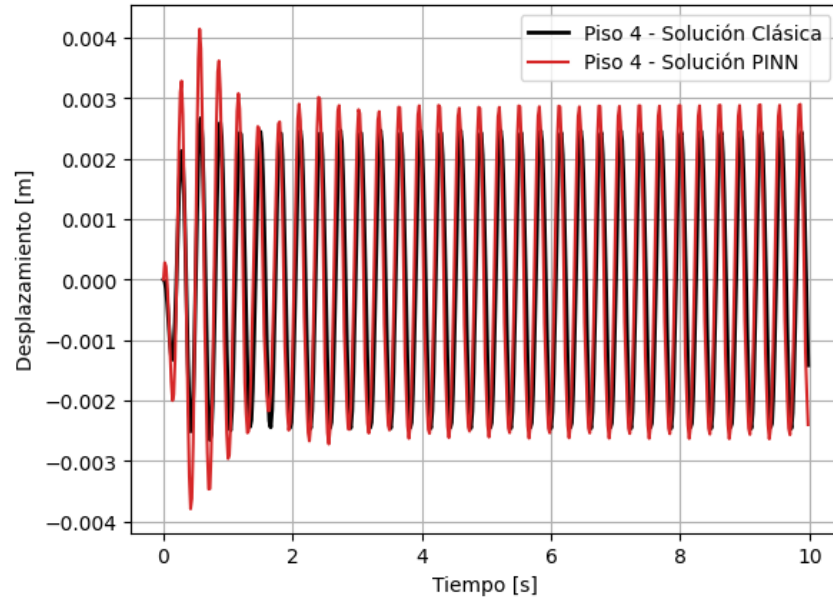


Figura 19: Resultado para la red en 4o piso con $\omega_{fuerza} = 3,225$ Hz (dentro de rango).
 $MAE_{piso} \approx 6,10 \cdot 10^{-4}$

4. Conclusiones y Trabajo Futuro (Fase 3)

Revisando el procedimiento llevado a cabo y los resultados obtenidos de este, se logra concluir un avance significativo en el desarrollo de una PINN como resolutor de ecuaciones diferenciales. Se logra desarrollar un modelo que, por medio del *Time Marching* logra propagar soluciones a tiempos elevados, sin fallar en puntos cruciales como coherencia y conservación de la energía, como puede verse en la figura 16. La red también logra producir soluciones que logran capturar la tendencia general del fenómeno para valores arbitrarios de ω_{fuerza} dentro del rango de entrenamiento, verificando la funcionalidad del concepto de *Parametric PINN*. De todas maneras no se logra hacer que el modelo tenga una convergencia del 100 % respecto a la solución *clásica* del sistema. Esto, junto al coste de desarrollo y evaluación órdenes de magnitud superiores (17) a alternativas numéricas tradicionales, plantean una baja aplicabilidad de este modelo específico. Dentro de los resultados más relevantes obtenidos durante este desarrollo se encuentran la efectividad altamente superior a opciones previamente exploradas, tanto de la arquitectura SIREN como del optimizador SOAP; ambos planteando mejoras de varios órdenes de magnitud en el sistema, en comparación a las implementaciones anteriores documentadas. Estos resultados incitan al uso de estas herramientas para proyectos futuros.

Considerando los objetivos planteados al inicio de esta segunda etapa, no logramos concretar la implementación de un modelo funcional que represente una mejora ante solvers clásicos. Bajo esta premisa los objetivos a plantear para la fase final caen en dos posibles líneas:

1. reimplementar el modelo en JAX, con el fin de probar el uso de SS-Broyden como optimizador, buscando obtener una mejor convergencia en la solución de la red.
2. Buscar plantear un modelo para el problema inverso, que por medio de las vibraciones resultantes de la estructura, poder identificar cambios en la matriz de amortiguamiento K , con el fin operativo de identificar fallos estructurales en tiempo real ante eventos como sismos.

Entre muchas opciones, la segunda se identifica mucho más interesante, planteando un paradigma al cual solvers clásicos no tienen acceso, a la vez de ser altamente relevante, si es que se cumple efectivamente la premisa de la representatividad estructural de este modelo sobre un edificio real. Cabe remarcar que esta línea de desarrollo podría no ser lograda dadas las limitaciones temporales para la tercera entrega, a la vez de representar un cambio de paradigma de estudio importante, posiblemente imposibilitando su realización.

Referencias

- [1] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*.
- [2] Khodakarami, S., Oommen, V., Daryakenari, N. A., Beekenkamp, M., & Karniadakis, G. E. (2026). *Spectral bias in physics-informed and operator learning: Analysis and mitigation guidelines*. *arXiv preprint arXiv:2602.19265*.
- [3] Rahaman, N., et al. (2019). On the spectral bias of neural networks. *International Conference on Machine Learning (ICML)*.
- [4] Sitzmann, V., Martel, J., Bergman, A., Lindell, D., & Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [5] Wang, S., Teng, Y., & Perdikaris, P. (2021). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*.
- [6] Vyas, N., et al. (2024). SOAP: Second-Order Approximation Preconditioner for Physics-Informed Neural Networks. *arXiv preprint*. repositorio oficial de SOAP
- [7] Wenhao Zhang, et. al. (2024). A General Method for Solving Differential Equations of Motion Using Physics-Informed Neural Networks.